



Fig. 2: Structure of SmartSoft and what it means to you

include clearly-defined semantics for interfacing these components. While the component based approach gives the robotic engineer flexibility in achieving software complexity, the catch is in having a system that can assist putting together a complete setup and providing software architecture without enforcing a specific robot architecture. It is precisely this fact that SmartSoft stands for.

Quoting from SmartSoft website, "The basic idea is to provide a small set of communication patterns, which can transmit objects between components and then squeeze every component interaction into those predefined patterns. Components interact solely via those patterns."

The entire process is segregated into components, communication patterns and objects, and services. Components are technically implemented as processes containing multiple threads and interacting via predefined communication patterns. Patterns define the communication mode, doing away with all communication and synchronisation issues, and acting like the master-slave protocol.

Content of the packet is the communication object that is always transmitted by value, to avoid intermittent inter-component communication while using other methods. Every time a packet is instantiated, a service is provided.

**Sensorimotor systems.** SmartSoft is built keeping in mind sensorimotor systems, but you can exploit the concept to suit any domain you choose. This tool looks at decoupling as the way of mastering the overall complexity of software of sensorimotor systems. You have templates for standardised communication patterns.

**From the bottom-up.** The main problem behind looking at things from a component perspective is the integration to form a whole. Many components might be manufactured concurrently and independently, and sometimes at different sites. How does one account for communication between these?

This is where the component interfaces come into the picture; these need to be powerful enough to decouple external behaviour from internal implementations, and this is what SmartSoft sets out to do. It allows you to focus on a single component, go into depth and perfect it, regardless of the internals of other components. You then simply compose a system out of these approved components.

The same is extended to components that are being reused, avoiding the need for designing or analysing from scratch. Internal state automation has at least three states representing neutral, alive and fatal states. Be it software or hardware, with lack of implementation standards at the former level, the bottom-up method turns advantageous.

**The wiring pattern.** The approach allows for dynamic wiring of components during run-time. Thus, loosely-coupled parts can easily be put together and adjusted according to the current context and environment, paving the way for designing distributed systems with ease. Control and data flows are configurable from outside a component.

**Asynchronicity, simplicity.** Decouple components to avoid passing on tight time dependencies and exploit concurrency to reduce latencies—these are the two mantras that are rigorously followed in any robot-

ics application, the base of which is asynchronicity accessed without involving the framework user.

Usage allowances of a tool is what decides if a robotics engineer can work on his or her application without being dependent on the software counterpart. SmartSoft takes this into consideration, addressing challenging topics like location transparency of components, synchronisation parameters or thread safety.

## Model-driven software development

SmartMDS Toolchain aims at providing a well-laid path from code-driven designs towards model-driven systems. The strict component based approach of SmartSoft separates middleware aspects from algorithmic solutions, paving the way for reuse and composability of software building blocks. Developers claim that special focus is put on service-oriented architectures for service robotics on top of quality of QoS middleware.

## Set up from scratch, or do it the easy way

You can choose to install SmartSoft package the code way, manual way or just copy VMWare image from this month's DVD and get on with it. The initial setup is straightforward—unpack and install VM package, followed by SmartSoft MDS Toolchain.

The team has put together tutorials that guide you right from taking the first step to running a pioneer 3DX robot. Another shows how to deploy a simple joystick navigation to RobotinoXT. They have also provided a collection of SmartSoft components that can be downloaded and used right away. Look for the component store at [www.servicerobotik-ulm.de/drupal/?q=node/81](http://www.servicerobotik-ulm.de/drupal/?q=node/81). Sourceforge also hosts an adaptive communication environment (ACE) based implementation for Windows and Linux. **REVIEW**

Priya Ravindran is M.Sc (electronics) from VIT University, Vellore, Tamil Nadu. She loves to explore new avenues and is passionate about writing